

NiceGUI ui.colors 完全指南（基于官方文档深度解析）

`ui.colors` 是 NiceGUI 提供的全局颜色管理工具，用于统一配置应用的主题颜色体系，支持内置主题切换、自定义颜色变量、动态主题更新等核心功能。其设计目标是**简化颜色一致性维护**，避免零散的样式定义，尤其适合需要统一视觉风格的中大型应用（如图书管理系统）。

一、核心定义与设计理念

1. 本质与定位

- `ui.colors(primary: str = None, secondary: str = None, ..., **kwargs) -> None`：通过关键字参数配置全局颜色变量，覆盖 NiceGUI 内置的颜色体系。
- 底层原理：修改应用的 CSS 变量（如 `--q-primary`、`--q-secondary`），所有依赖这些变量的组件（按钮、输入框、卡片等）会自动响应样式变化。
- 核心价值：
 - 全局统一：一次配置，所有组件共享颜色体系；
 - 动态切换：支持运行时修改颜色，实现主题切换功能；
 - 兼容 Tailwind：自定义颜色可与 Tailwind CSS 类结合使用；
 - 简化开发：无需手动为每个组件设置颜色类。

2. 与直接设置 `classes` 的区别

特性	<code>ui.colors</code> 方式	直接设置 <code>classes</code> 方式
作用范围	全局生效（所有依赖颜色变量的组件）	局部生效（仅当前组件）
颜色一致性	天然统一，无需重复定义	需手动确保类名一致，易出错
动态修改	支持运行时切换（一行代码更新全局）	需遍历所有组件修改类名，代码冗余
适配性	自动兼容 NiceGUI/Quasar 组件样式	需手动适配不同组件的样式结构
适用场景	应用主题配置、全局颜色调整	单个组件个性化颜色、局部样式差异

二、基础用法：内置颜色变量（官方核心配置项）

NiceGUI 基于 Quasar 框架，提供了 10+ 个核心颜色变量，覆盖绝大多数组件的默认样式。以下是官方文档明确支持的颜色变量及说明：

颜色变量名	作用说明	应用组件示例
<code>primary</code>	主色调（核心按钮、选中状态、强调文本）	按钮默认背景、输入框聚焦边框、标签高亮文本

颜色变量名	作用说明	应用组件示例
<code>secondary</code>	辅助色（次要按钮、提示信息、次要强调）	次要按钮背景、通知提示框、进度条
<code>accent</code>	强调色（特殊操作、重点标记、交互反馈）	特殊按钮、徽章、滑块激活状态
<code>positive</code>	成功色（成功提示、验证通过、正向反馈）	成功通知、勾选图标、通过状态标签
<code>negative</code>	错误色（错误提示、验证失败、负向反馈）	错误通知、删除按钮、警告边框
<code>info</code>	信息色（提示信息、说明文本、中性反馈）	信息通知、帮助图标、说明文本
<code>warning</code>	警告色（警告提示、风险操作、需注意状态）	警告通知、风险按钮、提醒标签
<code>dark</code>	深色（深色背景、深色文本、夜间模式）	深色卡片、夜间模式文本、深色边框
<code>light</code>	浅色（浅色背景、浅色文本、日间模式）	浅色卡片、日间模式背景、浅色边框
<code>background</code>	全局背景色（应用页面背景）	页面整体背景、容器背景
<code>surface</code>	表层色（卡片、面板、组件容器背景）	卡片背景、输入框背景、面板背景
<code>text</code>	文本主色（主要内容文本）	标签文本、按钮文本、输入框占位符
<code>text_secondary</code>	文本次要色（辅助说明文本）	说明文本、次要信息、禁用状态文本

基础配置示例（静态主题）

```
from nicegui import ui

# 配置全局主题颜色（图书管理系统示例：蓝色主调，专业稳重）
ui.colors(
    primary="#165DFF",          # 主色：蓝色（核心按钮、标题）
    secondary="#6B7280",        # 辅助色：灰色（次要操作、说明文本）
    positive="#00B42A",         # 成功色：绿色（添加成功、验证通过）
    negative="#F53F3F",         # 错误色：红色（删除操作、验证失败）
    info="#86909C",             # 信息色：浅灰色（提示信息）
    warning="#FF7D00",          # 警告色：橙色（警告提示、风险操作）
    background="#F7F8FA",       # 页面背景色：浅灰蓝
    surface="#FFFFFF",          # 表层色：白色（卡片、输入框背景）
    text="#1D2129",             # 文本主色：深灰色
    text_secondary="#86909C"    # 文本次要色：浅灰色
)

# 组件自动应用主题颜色（无需额外设置颜色类）
ui.label("图书管理系统").classes("text-2xl font-bold") # 文本主色
ui.button("添加图书") # 主色背景（primary）
```

```
ui.button("取消", secondary=True) # 辅助色背景 (secondary)
ui.button("删除", color="negative") # 错误色背景 (negative)
ui.input("搜索图书", placeholder="输入书名搜索") # 表层色背景+文本色
ui.notify("图书添加成功", type="positive") # 成功色通知

ui.run()
```

三、核心特性：动态主题切换

`ui.colors` 支持运行时动态修改颜色变量，结合 NiceGUI 的响应式变量，可实现一键主题切换（如日间 / 夜间模式）。

动态主题切换示例（日间 / 夜间模式）

```
from nicegui import ui

# 响应式变量控制主题模式
is_dark_mode = ui.reactive(False)

def toggle_theme():
    is_dark_mode.value = not is_dark_mode.value
    if is_dark_mode.value:
        # 夜间模式：深色背景、浅色文本
        ui.colors(
            primary="#3B82F6",
            background="#1E293B",
            surface="#334155",
            text="#F8FAFC",
            text_secondary="#94A3B8",
            secondary="#64748B"
        )
    else:
        # 日间模式：浅色背景、深色文本
        ui.colors(
            primary="#165DFF",
            background="#F7F8FA",
            surface="FFFFFF",
            text="#1D2129",
            text_secondary="#86909C"
        )

# 初始主题配置
toggle_theme()

# 主题切换控件
ui.switch("夜间模式", value=is_dark_mode, on_change=toggle_theme)

# 测试组件
ui.label("图书管理系统").classes("text-2xl font-bold")
with ui.card().classes("w-64 p-6"):
    ui.label("热门图书").classes("font-bold mb-2")
    ui.text("《Python编程：从入门到实践》")
```

```
ui.text("作者: 埃里克·马瑟斯") # 文本次要色
```

```
ui.button("添加图书")
ui.button("查看详情", secondary=True)
```

```
ui.run()
```

四、高级用法：自定义颜色与扩展

1. 颜色值格式支持

`ui.colors` 支持多种颜色格式，完全兼容 CSS 颜色标准：

- 十六进制（推荐）：`#165DFF`（6 位）、`#F00`（3 位缩写）
- RGB/RGBA：`rgb(22, 93, 255)`、`rgba(22, 93, 255, 0.8)`（透明度）
- HSL/HSLA：`hsl(220, 100%, 56%)`、`hsla(220, 100%, 56%, 0.8)`
- 颜色名称：`blue`、`red`、`green`（CSS 标准颜色名）

示例：

```
ui.colors(
    primary="rgba(22, 93, 255, 0.9)", # 带透明度的主色
    secondary="hsl(200, 10%, 40%)",   # HSL 格式辅助色
    warning="orange"                  # 颜色名称
)
```

2. 结合 Tailwind CSS 使用

自定义的颜色变量可通过 Tailwind 的 `theme.extend` 扩展为 Tailwind 类，实现全局颜色与原子类的统一：

```
from nicegui import ui

# 1. 配置全局颜色变量
ui.colors(
    primary="#165DFF",
    secondary="#6B7280"
)

# 2. 扩展 Tailwind 主题，引用全局颜色变量
ui.add_head_html('''
<script>
    tailwind.config = {
        theme: {
            extend: {
                colors: {
                    // 引用 NiceGUI 全局颜色变量
                    primary: 'var(--q-primary)',
                    secondary: 'var(--q-secondary)',
                },
            },
        },
    }
</script>
''')
```

```

    }
}
</script>
<style type="text/tailwindcss">
  @layer components {
    .btn-primary {
      @apply bg-primary text-white px-4 py-2 rounded-md hover:bg-primary/90;
    }
    .text-primary {
      @apply text-primary font-medium;
    }
  }
</style>
'''

```

```

# 3. 使用扩展后的 Tailwind 类（与全局主题保持一致）
ui.label("图书标题").classes("text-primary text-xl")
ui.button("添加图书").classes("btn-primary")
ui.card().classes("border border-secondary/20 p-4")

ui.run()

```

3. 组件级颜色覆盖（局部差异）

若需在全局主题基础上修改单个组件颜色，可通过以下方式实现（优先级：组件自身配置 > 全局 `ui.colors`）：

```

# 1. 按钮通过 color 参数覆盖（支持全局颜色变量名）
ui.button("自定义颜色按钮", color="primary") # 使用全局 primary 色
ui.button("红色按钮", color="negative")      # 使用全局 negative 色

# 2. 通过 classes 覆盖（优先级最高）
ui.button("橙色按钮").classes("bg-orange-500 text-white") # 忽略全局主题

# 3. 输入框通过 focus 样式覆盖
ui.input("自定义聚焦颜色").classes("focus:border-orange-500")

```

4. 批量修改颜色变量（多主题切换）

可定义多个主题配置，通过按钮切换不同风格（如图书管理系统的「默认主题」「专业主题」「活力主题」）：

```

from nicegui import ui

# 定义主题配置字典
themes = {
    "default": {
        "primary": "#165DFF",
        "background": "#F7F8FA",
        "surface": "#FFFFFF",
        "text": "#1D2129"
    },
    "professional": {
        "primary": "#0F172A",

```

```

        "background": "#F1F5F9",
        "surface": "#FFFFFF",
        "text": "#0F172A"
    },
    "vibrant": {
        "primary": "#8B5CF6",
        "background": "#FAFAFA",
        "surface": "#FFFFFF",
        "text": "#111827"
    }
}

def switch_theme(theme_name):
    ui.colors(**themes[theme_name])
    ui.notify(f"已切换至 {theme_name} 主题")

# 初始主题
ui.colors(**themes["default"])

# 主题切换按钮组
with ui.row().classes("gap-2 p-4"):
    ui.button("默认主题", on_click=lambda: switch_theme("default"))
    ui.button("专业主题", on_click=lambda: switch_theme("professional"))
    ui.button("活力主题", on_click=lambda: switch_theme("vibrant"))

# 测试组件
ui.label("图书管理系统").classes("text-2xl font-bold")
ui.card().classes("w-64 p-6").add(ui.text("图书列表"))
ui.button("添加图书")

ui.run()

```

五、官方文档重点强调的注意事项

1. 颜色变量优先级

- 组件自身 `color` 参数 > 组件 `classes` 颜色类 > 全局 `ui.colors` 配置 > NiceGUI 默认颜色；
- 若需强制覆盖组件颜色，建议使用 `classes` 并添加 `!important`（如 `!bg-red-500`）。

2. 兼容性说明

- `ui.colors` 主要影响基于 Quasar 的组件（如按钮、输入框、卡片、通知），自定义 HTML 元素需手动引用 CSS 变量（如 `background: var(--q-primary)`）；
- 部分第三方组件可能不兼容 NiceGUI 的颜色变量，需单独适配。

3. 透明度与对比度

- 推荐使用 RGBA/HSLA 格式设置带透明度的颜色，避免直接修改 `opacity`（会影响元素内所有内容）；
- 确保文本与背景色的对比度符合 WCAG 标准（如文本色与背景色对比度 $\geq 4.5:1$ ），提升可访问性。

4. 性能考虑

- 动态切换主题时，`ui.colors` 会批量更新 CSS 变量，性能优于遍历组件修改类名；
- 避免频繁调用 `ui.colors`（如每秒多次切换），可能导致 UI 闪烁。

5. 与 `ui.add_css` 的配合

- `ui.colors` 用于配置全局颜色变量，`ui.add_css` 用于定义基于这些变量的样式规则；
- 示例：

```
ui.colors(primary="#165DFF")
ui.add_css('''
    .custom-card {
        border: 1px solid var(--q-primary);
        background-color: var(--q-background);
    }
''')
ui.card().classes("custom-card")
```

六、图书管理系统实战场景示例

场景 1：基于角色的主题切换（管理员 / 普通用户）

```
from nicegui import ui

def set_role_theme(role):
    if role == "admin":
        # 管理员主题：红色主调（权威、醒目）
        ui.colors(
            primary="#F53F3F",
            background="#F7F8FA",
            surface="FFFFFF",
            text="#1D2129"
        )
    else:
        # 普通用户主题：蓝色主调（友好、专业）
        ui.colors(
            primary="#165DFF",
            background="#F7F8FA",
            surface="FFFFFF",
            text="#1D2129"
        )

# 模拟角色选择
with ui.row().classes("gap-2 p-4"):
    ui.button("管理员模式", on_click=lambda: set_role_theme("admin"))
    ui.button("普通用户模式", on_click=lambda: set_role_theme("user"))

# 系统核心组件
ui.label("图书管理系统").classes("text-2xl font-bold")
```

```

with ui.card().classes("w-80 p-6"):
    ui.label("图书管理菜单").classes("font-bold mb-4")
    ui.button("添加图书")
    ui.button("删除图书", color="negative")
    ui.button("修改图书信息", secondary=True)

ui.run()

```

场景 2：图书状态标签颜色配置

```

from nicegui import ui

# 配置图书状态相关颜色
ui.colors(
    available="#00B42A",    # 可借阅：绿色
    borrowed="#FF7D00",     # 已借出：橙色
    reserved="#F53F3F",     # 已预约：红色
    overdue="#F53F3F",      # 逾期：红色
    maintenance="#86909C"  # 维护中：灰色
)

# 扩展 Tailwind 类，用于状态标签
ui.add_head_html('''
<style type="text/tailwindcss">
    @layer components {
        .status-available {
            @apply bg-available/10 text-available px-2 py-1 rounded text-sm;
        }
        .status-borrowed {
            @apply bg-borrowed/10 text-borrowed px-2 py-1 rounded text-sm;
        }
        .status-reserved {
            @apply bg-reserved/10 text-reserved px-2 py-1 rounded text-sm;
        }
    }
</style>
''')

# 图书列表（带状态标签）
books = [
    {"title": "Python编程：从入门到实践", "status": "available"},
    {"title": "数据结构与算法", "status": "borrowed"},
    {"title": "机器学习实战", "status": "reserved"}
]

with ui.card().classes("w-96 p-4"):
    ui.label("图书列表").classes("text-xl font-bold mb-4")
    for book in books:
        with ui.row().classes("justify-between items-center mb-2"):
            ui.text(book["title"])
            if book["status"] == "available":
                ui.label("可借阅").classes("status-available")

```



```
elif book["status"] == "borrowed":
    ui.label("已借出").classes("status-borrowed")
elif book["status"] == "reserved":
    ui.label("已预约").classes("status-reserved")
```

```
ui.run()
```

七、总结

`ui.colors` 是 NiceGUI 中管理全局颜色的核心工具，其核心优势在于**统一配置、动态切换、兼容扩展**。在图书管理系统开发中，合理使用 `ui.colors` 可实现：

1. 全局视觉风格统一（如主色调、文本色、背景色）；
2. 灵活的主题切换（如日间 / 夜间模式、角色专属主题）；
3. 组件样式一致性（无需重复设置颜色类）；
4. 易维护性（修改颜色仅需更新 `ui.colors` 配置）。

结合 Tailwind CSS 和 `ui.add_css`，可进一步扩展颜色的使用场景，实现从全局主题到局部组件的精细化样式控制。